

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО - КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ**

**Дисциплина «Конструирование программного обеспечения для
систем искусственного интеллекта»**

**Отчет
по лабораторной работе №1
на тему: «Исследование методов поиска в пространстве состояний»**

Выполнил: студент группы ПИЖ-б-о-21-1
Трушева Вероника Олеговна

(подпись)

Проверил: доцент департамента цифровых,
робототехнических систем и электроники
института перспективной инженерии
Воронкин Роман Александрович

(подпись)

Ставрополь, 2024 г.

Цель работы: приобретение навыков по работе с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x.

Ход работы

Ссылка на гитхаб: https://github.com/pynikcmd/1_DS_AI

1. Изучить теоретический материал работы.
2. Создать общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT и язык программирования Python.

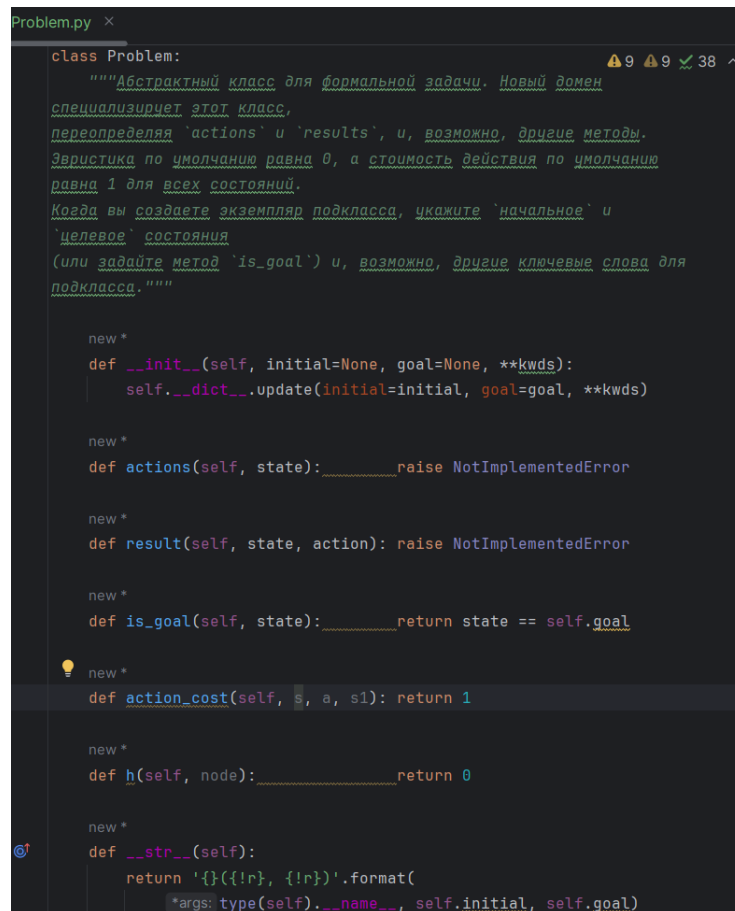
Рисунок 1 – Создание репозитория

3. Выполните клонирование созданного репозитория.

```
D:\4 курс\Конструирование программного обеспечения для систем искусственного интеллекта>git clone https://github.com/pynikcmd/1_DS_AI.git
Cloning into '1_DS_AI'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 5.02 KiB | 2.51 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

Рисунок 2 – Клонирование репозитория

4. Дополните файл .gitignore необходимыми правилами для работы с IDE PyCharm либо Visual Studio Code
5. Создайте проект Python в папке репозитория.
6. Проработайте примеры лабораторной работы. Создайте для каждого примера отдельный модуль языка Python. Зафиксируйте изменения в репозитории.



```
class Problem:
    """Абстрактный класс для формальной задачи. Новый домен
    специализирует этот класс,
    переопределяя 'actions' и 'results', и, возможно, другие методы.
    Эвристика по умолчанию равна 0, а стоимость действия по умолчанию
    равна 1 для всех состояний.
    Когда вы создаете экземпляр подкласса, укажите 'начальное' и
    'целевое' состояния
    (или задайте метод 'is_goal') и, возможно, другие ключевые слова для
    подкласса."""

    new *
    def __init__(self, initial=None, goal=None, **kwargs):
        self.__dict__.update(initial=initial, goal=goal, **kwargs)

    new *
    def actions(self, state): raise NotImplementedError

    new *
    def result(self, state, action): raise NotImplementedError

    new *
    def is_goal(self, state): return state == self.goal

    new *
    def action_cost(self, s, a, s1): return 1

    new *
    def h(self, node): return 0

    new *
    def __str__(self):
        return '{}({!r}, {!r})'.format(
            *args: type(self).__name__, self.initial, self.goal)
```

Рисунок 3 – Код программы

```

9 class Node:
10     """Узел в дереве поиска"""
11     new *
12     def __init__(self, state, parent=None, action=None, path_cost=0):
13         self.__dict__.update(state=state, parent=parent, action=action,
14                               path_cost=path_cost)
15     new *
16     def __repr__(self): return '<{}>'.format(self.state)
17     new *
18     def __len__(self): return 0 if self.parent is None else (1 +
19                                                             len(self.parent))
20     new *
21     def __lt__(self, other): return self.path_cost < other.path_cost
22
23 failure = Node(state='failure', path_cost=math.inf) # Алгоритм не смог найти ре
24 cutoff = Node(state='cutoff', path_cost=math.inf) # Указывает на то, что поиск с
25 new *
26 def expand(problem, node):
27     """Раскрываем узел, создав дочерние узлы."""
28     s = node.state
29     for action in problem.actions(s):
30         s1 = problem.result(s, action)
31         cost = node.path_cost + problem.action_cost(s, action, s1)
32         yield Node(s1, node, action, cost)
33 1 usage new *
34 def path_actions(node):
35     """Последовательность действий, чтобы добраться до этого узла."""
36     if node.parent is None:
37         return []
38     return path_actions(node.parent) + [node.action]
39 1 usage new *
40 def path_states(node):
41     """Последовательность состояний, чтобы добраться до этого узла"""
42     if node in (cutoff, failure, None):

```

Рисунок 4 – Код программы

```

4 import sys
5 from collections import defaultdict, deque, Counter
6 from itertools import combinations
7
8
9 FIFOQueue = deque
10 LIFOQueue = list
11 new *
12 class PriorityQueue:
13     """Очередь, в которой элемент с минимальным значением f(item) всегда
14     выгружается первым."""
15     new *
16     def __init__(self, items=(), key=lambda x: x):
17         self.key = key
18         self.items = [] # a heap of (score, item) pairs
19         for item in items:
20             self.add(item)
21 1 usage new *
22 def add(self, item):
23     """Добавляем элемент в очередь."""
24     pair = (self.key(item), item)
25     heapq.heappush(self.items, pair)
26 new *
27 def pop(self):
28     """Достаем и возвращаем элемент с минимальным значением
29     f(item)."""
30     return heapq.heappop(self.items)[1]
31 new *
32 def top(self): return self.items[0][1]
33 new *
34 def __len__(self): return len(self.items)

```

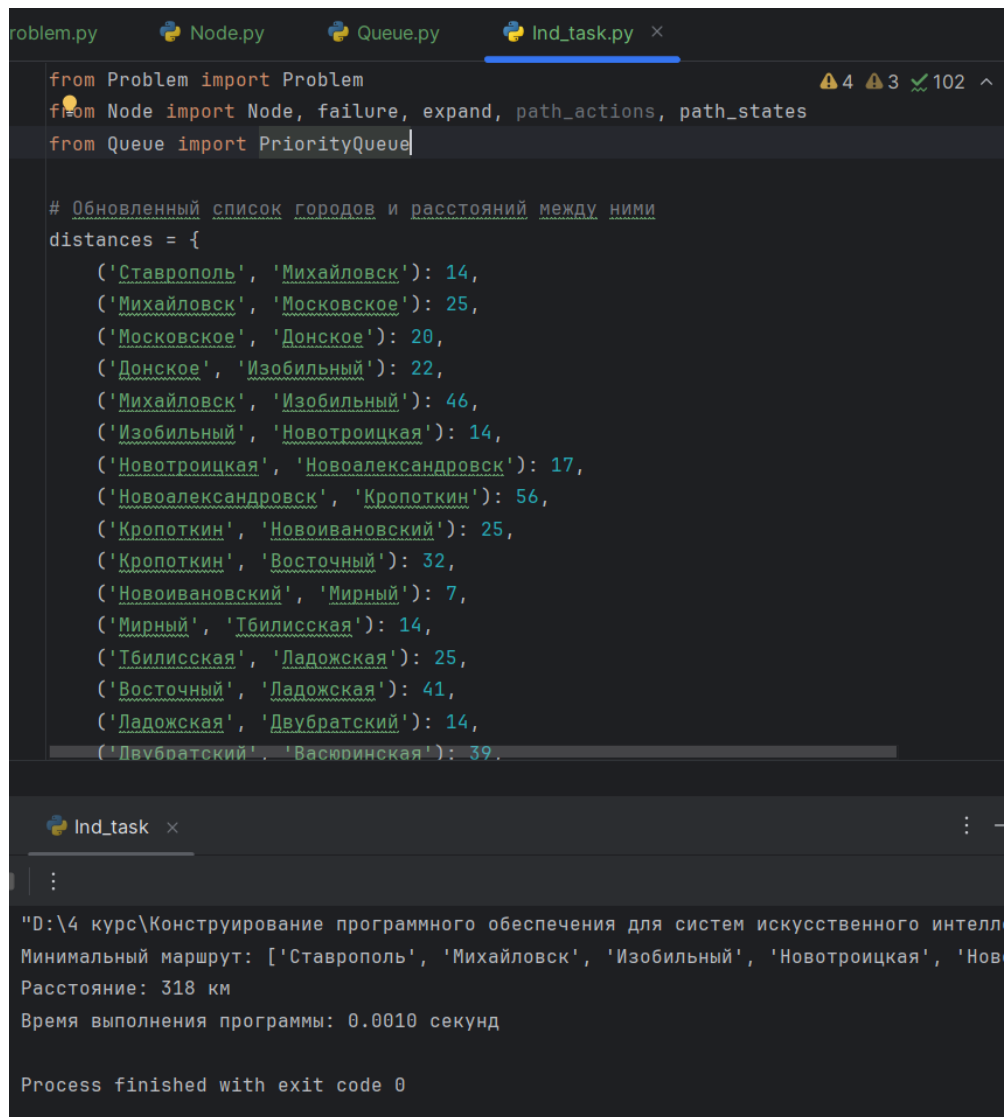
Рисунок 5 – Код программы

7. Воспользуйтесь сервисом Google Maps или аналогичными картографическими приложениями, чтобы выбрать на карте более 20 населённых пунктов, связанных между собой дорогами. Постройте граф, где

8. Методом полного перебора решите [задачу коммивояжёра]
(<https://ru.wikipedia.org/wiki/%D0%97%D0%B0%D0%B4%D0%B0%D1%87%>



начального населенного пункта на построенном графе. Оцените время решения данной задачи, если оно окажется достаточно большим, уменьшите количество узлов и ребер графа. Покажите полученный путь на построенном графе.



```
from Problem import Problem
from Node import Node, failure, expand, path_actions, path_states
from Queue import PriorityQueue

# Обновленный список городов и расстояний между ними
distances = {
    ('Ставрополь', 'Михайловск'): 14,
    ('Михайловск', 'Московское'): 25,
    ('Московское', 'Донское'): 20,
    ('Донское', 'Изобильный'): 22,
    ('Михайловск', 'Изобильный'): 46,
    ('Изобильный', 'Новотроицкая'): 14,
    ('Новотроицкая', 'Новоалександровск'): 17,
    ('Новоалександровск', 'Кропоткин'): 56,
    ('Кропоткин', 'Новоивановский'): 25,
    ('Кропоткин', 'Восточный'): 32,
    ('Новоивановский', 'Мирный'): 7,
    ('Мирный', 'Тбилисская'): 14,
    ('Тбилисская', 'Ладжская'): 25,
    ('Восточный', 'Ладжская'): 41,
    ('Ладжская', 'Двубратский'): 14,
    ('Двубратский', 'Васюринская'): 39,
}
```

Output:

```
"D:\4 курс\Конструирование программного обеспечения для систем искусственного интелл
Минимальный маршрут: ['Ставрополь', 'Михайловск', 'Изобильный', 'Новотроицкая', 'Нов
Расстояние: 318 км
Время выполнения программы: 0.0010 секунд
Process finished with exit code 0
```

Рисунок – Код программы и результат ее работы

9. Зафиксируйте сделанные изменения в репозитории.
10. Добавьте отчет по лабораторной работе в формате PDF в папку doc репозитория. Зафиксируйте изменения.
11. Выполните слияние ветки для разработки с веткой main / master.
12. Отправьте сделанные изменения на сервер GitHub.
13. Отправьте адрес репозитория GitHub на электронный адрес преподавателя.

Вывод: в ходе выполнения лабораторной работы были изучены работы с методами поиска в пространстве состояний с помощью языка программирования Python версии 3.x.

Ответы на контрольные вопросы

1. Что представляет собой метод "слепого поиска" в искусственном интеллекте?

Метод «слепого поиска» в ИИ – это способ поиска решения без какой-либо дополнительной информации о том, насколько каждое действие приближает к цели. Он исследует все возможные варианты в поиске решения, часто используя подход проб и ошибок.

2. Как отличается эвристический поиск от слепого поиска?

Эвристический поиск использует дополнительные знания (эвристики) для направления поиска, что делает его более эффективным. В отличие от слепого поиска, который исследует все возможные варианты, эвристический поиск нацелен на минимизацию числа шагов за счет использования предположений и эвристических правил.

3. Какую роль играет эвристика в процессе поиска?

Эвристика помогает оценивать, насколько состояние или действие приближает к цели. Это позволяет алгоритму направлять процесс поиска более эффективно, отсекая менее перспективные пути.

4. Приведите пример применения эвристического поиска в реальной задаче.

Один из примеров – алгоритм A^* , который используется для поиска кратчайшего пути на картах. Эвристика здесь может представлять собой прямое расстояние до цели, помогая алгоритму находить оптимальный путь.

5. Почему полное исследование всех возможных ходов в шахматах затруднительно для ИИ?

В шахматах огромное количество возможных ходов и вариантов развития событий. Полное исследование всех ходов потребовало бы огромных

вычислительных ресурсов и времени, что делает его непрактичным даже для суперкомпьютеров.

6. Какие факторы ограничивают создание идеального шахматного ИИ?

Ограниченные вычислительные ресурсы (время и память) затрудняют анализ всех возможных вариантов игры. Кроме того, шахматы включают множество стратегий и тактик, которые необходимо учитывать.

7. В чем заключается основная задача искусственного интеллекта при выборе ходов в шахматах?

Основная задача ИИ – выбрать ход, который максимизирует его шансы на выигрыш, минимизируя шансы соперника.

8. Как алгоритмы ИИ балансируют между скоростью вычислений и нахождением оптимальных решений?

Алгоритмы ИИ используют эвристики для сокращения пространства поиска и нахождения хороших решений быстрее, при этом жертвуя некоторой точностью ради производительности.

9. Каковы основные элементы задачи поиска маршрута по карте?

Основные элементы включают состояния (города), действия (перемещения между городами), а также стоимость (расстояние между городами).

10. Как можно оценить оптимальность решения задачи маршрутизации на карте Румынии?

Оптимальность оценивается по минимальной стоимости пути, выраженной в километрах, между исходным городом (Арад) и целью (Бухарест).

11. Что представляет собой исходное состояние дерева поиска в задаче маршрутизации по карте Румынии?

Исходное состояние – это начальная точка (город Арад), с которой начинается исследование возможных маршрутов.

12. Какие узлы называются листовыми в контексте алгоритма поиска по дереву?

Листовые узлы – это узлы, которые еще не были расширены, то есть не исследованы для создания новых дочерних узлов.

13. Что происходит на этапе расширения узла в дереве поиска?

На этом этапе из текущего узла генерируются новые узлы, представляющие все возможные действия из текущего состояния.

14. Какие города можно посетить, совершив одно действие из Арада в примере задачи поиска по карте?

Из Арада можно отправиться в Сибиу, Тимишоару или Зеринд.

15. Как определяется целевое состояние в алгоритме поиска по дереву?

Целевое состояние определяется заранее как конечная точка поиска, например, город Бухарест.

16. Какие основные шаги выполняет алгоритм поиска по дереву?

- Инициализация.
- Выбор узла.
- Проверка на целевое состояние.
- Расширение узла.
- Продолжение цикла до достижения цели или неудачи

17. Чем различаются состояния и узлы в дереве поиска?

Состояние описывает текущее положение в пространстве решений, а узел хранит дополнительную информацию, такую как родительский узел, действие и стоимость пути.

18. Что такое функция преемника и как она используется в алгоритме поиска?

Функция преемника генерирует все возможные действия из текущего состояния, создавая новые состояния для дальнейшего исследования.

19. Какое влияние на поиск оказывают такие параметры, как b (разветвление), d (глубина решения) и m (максимальная глубина)?

b (коэффициент разветвления) влияет на количество новых узлов, которые могут быть созданы из одного узла.

d (глубина решения) показывает, насколько далеко нужно углубиться для нахождения решения.

m (максимальная глубина) ограничивает количество уровней дерева

20. Как алгоритмы поиска по дереву оцениваются по критериям полноты, временной и пространственной сложности, а также оптимальности?

Оцениваются по полноте (способности найти решение), временной и пространственной сложности (количество узлов и памяти) и оптимальности (нахождение лучшего решения).

21. Какую роль выполняет класс `Problem` в приведенном коде?

Класс `Problem` определяет основную структуру задачи поиска, включая начальное и целевое состояние, возможные действия и их результаты

22. Какие методы необходимо переопределить при наследовании класса `Problem`?

Необходимо переопределить методы `actions`, `result`, и возможно, `action_cost` и `is_goal`, чтобы описать конкретную задачу.

23. Что делает метод `is_goal` в классе `Problem`?

Этот метод проверяет, достигнуто ли целевое состояние, возвращая `True` или `False`.

24. Для чего используется метод `action_cost` в классе `Problem`?

Этот метод возвращает стоимость выполнения действия между двумя состояниями.

25. Какую задачу выполняет класс `Node` в алгоритмах поиска?

Класс `Node` представляет узел дерева поиска, включая текущее состояние, родительский узел, действие, которое привело к этому узлу, и стоимость пути.

26. Какие параметры принимает конструктор класса `Node`?

Конструктор принимает параметры: текущее состояние (`state`), родительский узел (`parent`), действие (`action`) и стоимость пути (`path_cost`).

27. Что представляет собой специальный узел `failure`?

Этот узел указывает на неудачу поиска, когда решение не найдено.

28. Для чего используется функция `expand` в коде?

Расширяет узел, генерируя дочерние узлы для каждого возможного действия.

29. Какая последовательность действий генерируется с помощью функции `path_actions`?

Возвращает список действий, которые привели к данному узлу.

30. Чем отличается функция `path_states` от функции `path_actions`?

`path_states` возвращает последовательность состояний, тогда как `path_actions` – последовательность действий.

31. Какой тип данных используется для реализации `FIFOQueue`?

Для реализации очереди используется двусторонняя очередь deque из модуля collections.

32. Чем отличается очередь FIFOQueue от LIFOQueue?

FIFOQueue (First In, First Out) – это очередь, где первый добавленный элемент будет извлечен первым. Она используется, когда важен порядок обработки элементов в том порядке, в котором они поступили. Реализуется с помощью структуры deque из модуля collections.

LIFOQueue (Last In, First Out) – это стек, где последний добавленный элемент будет извлечен первым. В этом случае для реализации используется стандартный список Python list. Это подходит для задач, где последний добавленный элемент должен быть обработан первым (например, в алгоритмах поиска в глубину).

33. Как работает метод add в классе PriorityQueue?

Метод add добавляет элемент в очередь с приоритетом. Он принимает элемент, который добавляется в очередь вместе с его приоритетом. Приоритет определяется с помощью функции, переданной при инициализации очереди (по умолчанию используется значение самого элемента). Добавление выполняется с использованием функции heapq.heappush, которая добавляет элемент в кучу, сохраняя структуру данных в порядке согласно приоритетам.

34. В каких ситуациях применяются очереди с приоритетом?

Очереди с приоритетом применяются в ситуациях, где важно обрабатывать элементы не в порядке их поступления, а в зависимости от приоритета. Примеры:

Алгоритмы поиска на графах (например, A*).

Планирование задач, где задачи с более высоким приоритетом обрабатываются первыми.

Алгоритмы для решения задач с очередями ожидания (например, маршрутизация сетевого трафика или планирование процессов в ОС).

35. Как функция `heapq.heappop` помогает в реализации очереди с приоритетом?

Функция `heapq.heappop` извлекает элемент с наименьшим значением приоритета из очереди с приоритетом (кучи). Она поддерживает структуру кучи, так что после извлечения элемента оставшиеся элементы остаются в правильном порядке. Эта функция критически важна для работы очереди с приоритетом, так как гарантирует, что элемент с наименьшим приоритетом всегда будет извлечен первым.